Servant.Client: https://hackage.haskell.org/package/servant-client-0.19
Servant.Client.Core: https://hackage.haskell.org/package/servant-client-core

Servant.Client

```
-- Servant.Client
data ClientM a
client :: HasClient ClientM api => Proxy api -> ClientM
a ->
runClientM :: ClientM a -> ClientEnv -> IO (Either
ClientError a)

data ClientEnv = ClientEnv {
  manager :: Manager,
             -- Network.HTTP.Client.Manager
  baseUrl :: BaseUrl,
  cookieJar :: Maybe (TVar CookieJar),
  makeClientRequest :: BaseUrl -> Request -> Request
}
mkClientEnv :: Manager -> BaseUrl -> ClientEnv
```

Servant.Client.Core

```
clientIn :: HasClient m api => Proxy api -> Proxy m ->
Client m api
class RunClient m => HasClient m api where
  type Client (m :: * -> *) (api :: *) :: *
  clientWithRoute :: Proxy m -> Proxy api -> Request ->
Client m api
  hoistClientMonad :: Proxy m -> Proxy api -> (forall x.
mon x -> mon' x) -> Client mon api -> Client mon' api

foldMapUnion :: forall c a (as :: [Type]). All c as =>
Proxy c -> (forall x. c x => x -> a) -> Union as -> a
matchUnion :: forall a (as :: [Type]). IsMember a as =>
Union as -> Maybe a

class Monad m => RunClient m where
  runRequestAcceptStatus :: Maybe [Status] -> Request ->
m Response
  throwClientError :: ClientError -> m a

-- Settings
data BaseUrl = BaseUrl {
  baseUrlScheme :: Scheme
  baseUrlHost :: String
  baseUrlPort :: Int
  baseUrlPath :: String
}

data Scheme = Http | Https
showBaseUrl :: BaseUrl -> String
```

```
parseBaseUrl :: MonadThrow m => String -> m BaseUrl

newtype InvalidBaseUrlException =
InvalidBaseUrlException String
```

Request

```
-- Servant.Client.Core
type Request = RequestF RequestBody Builder
data RequestF body path = Request {
  requestPath :: path
  requestQueryString :: Seq QueryItem
  requestBody :: Maybe (body, MediaType)
  requestAccept :: Seq MediaType
  requestHeaders :: Seq Header
  requestHttpVersion :: HttpVersion
  requestMethod :: Method
}

defaultRequest :: Request
data RequestBody
  = RequestBodyLBS ByteString
  | RequestBodyBS ByteString
  | RequestBodySource (SourceIO ByteString)
mkAuthenticatedRequest :: AuthClientData a ->
(AuthClientData a -> Request -> Request) ->
AuthenticatedRequest a
basicAuthReq :: BasicAuthData -> Request -> Request
newtype AuthenticatedRequest a = AuthenticatedRequest {
  unAuthReq :: (AuthClientData a, AuthClientData a ->
Request -> Request)
}

addHeader :: ToHttpApiData a => HeaderName -> a ->
Request -> Request
appendToQueryString :: Text -> Maybe ByteString ->
Request -> Request
appendToPath :: Builder -> Request -> Request
setRequestBodyLBS :: ByteString -> MediaType -> Request
-> Request
setRequestBody :: RequestBody -> MediaType -> Request ->
Request
encodeQueryParamValue :: ToHttpApiData a => a ->
ByteString
```

Response

```
-- Servant.Client.Core
type Response = ResponseF ByteString
data ResponseF a = Response {
  responseStatusCode :: Status
  responseHeaders :: Seq Header
  responseHttpVersion :: HttpVersion
```

```
  responseBody :: a
}

-- Error
type family AuthClientData a :: *
data ClientError
  = FailureResponse (RequestF () (BaseUrl, ByteString))
Response
  | DecodeFailure Text Response
  | UnsupportedContentType MediaType Response
  | InvalidContentTypeHeader Response
  | ConnectionError SomeException
data EmptyClient = EmptyClient
```

Example: Basic usage
e.g.

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeOperators #-}

import Network.HTTP.Client hiding (Proxy)
import Data.Proxy
import Servant
import Servant.Client

serverUrl :: BaseUrl
serverUrl = BaseUrl Http "localhost" 1234 "/"

run :: ClientM a -> IO (Either ClientError a)
run cl = do
  manager <- newManager defaultManagerSettings
  runClientM cl (mkClientEnv manager serverUrl)


-- API
type MYAPI = "api" :>
  (QueryParam' '[Required] "name" String :> Get '[JSON]
String
  :<|> ReqBody '[JSON] [Int] :> Post '[JSON] ())

getReq :: String -> ClientM String
createReq :: [Int] -> ClientM ()
getReq :<|> createReq = client (Proxy @MYAPI)
```

use

```
test :: IO ()
test = do
  result <- run getReq
  case result of
   Left e -> putStrLn $ "error:" <> show e
   Right s -> putSTrLn $ "success:" <> s
```

Example: create a non-existent API client
Just make a random API type

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE TypeOperators #-}

import Data.Proxy
import Servant
import Servant.Client

someRandomClient :: ClientM ()
someRandomClient = client (Proxy @("badroute" :> Get
'[JSON] ()))
```

```
-- Basic usage
--
-- case: verb
type API = Get '[JSON] String
call :: ClientM String
call = client (Proxy @API)

-- case: sum
-- Use pattern matching
type API = Get '[JSON] String :<|> Post '[JSON] ()
get :: ClientM String
     -- GET
post :: ClientM ()
     -- POST
get :<|> post = client (Proxy @API)

-- case: sum in inner
type API = Capture "uid" String :>
  (Get '[JSON] String :<|> Post '[JSON] ())
call = client (Proxy @API)
get :: String -> ClientM String
  -- GET
get uid = f
  where (f :<|> _) = call uid
post :: String -> ClientM ()
   -- POST
post uid = f
  where (f :<|> _) = call uid


-- case: query param
type API = QueryParam' '[Required] "name" String :>
Get '[JSON] String
 :<|> QueryParam' '[Optional] "name" String :>
Get '[JSON] String
call1 :: String -> ClientM String
```

```
   -- Required
call2 :: Maybe String -> ClientM String
   -- Optional
call1 :<|> client2 = client (Proxy @API)

-- case: capture
type API = Capture "uid" String :> Get '[JSON] String
call :: String -> ClientM String
call = client (Proxy @API)

-- case: request body
type API = ReqBody '[JSON] String :> Post '[JSON] ()
call :: String -> ClientM ()
call = client (Proxy @API)
```

```
============================================================
                            ======
```

```
This
is
code
Monaco 14
```